

UNIT 7: GRAPH OPERATIONS AND TREES

Operations, Tree Properties, and Computational Analysis

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-II



Course Outline

- 1 Operations on Graphs
- 2 Trees as Graphs
- 3 Algorithms and Analysis
- 4 Summary



Outline

- 1 Operations on Graphs
- 2 Trees as Graphs
- 3 Algorithms and Analysis
- 4 Summary



Basic Graph Operations

Set-Theoretic Operations

Graph operations **manipulate** graph structure through *set operations* on vertices and edges.

Union

Combine two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ into $G = (V_1 \cup V_2, E_1 \cup E_2)$. **Result:** All vertices and edges from *both* graphs.

Intersection

Retain **only common** vertices and edges: $G = (V_1 \cap V_2, E_1 \cap E_2)$. **Result:** *Shared subgraph* structure.



Basic Graph Operations

Set-Theoretic Operations

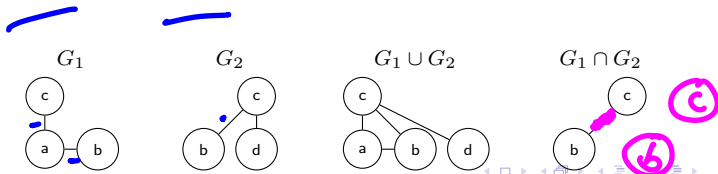
Graph operations **manipulate** graph structure through *set operations* on vertices and edges.

Union

Combine two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ into $G = (V_1 \cup V_2, E_1 \cup E_2)$. **Result:** All vertices and edges from *both* graphs.

Intersection

Retain **only common** vertices and edges: $G = (V_1 \cap V_2, E_1 \cap E_2)$. **Result:** *Shared subgraph* structure.



Advanced Graph Operations

Ring Sum (*Symmetric Difference*)

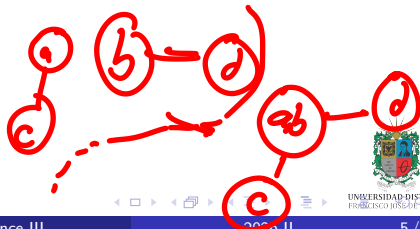
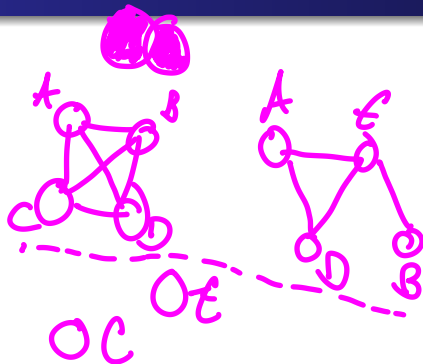
Combine edges **not shared** by both graphs: $E = (E_1 \cup E_2) \setminus (E_1 \cap E_2)$.

Vertex and Edge Deletion

- **Delete vertex** v : Remove v and all incident edges.
- **Delete edge** e : Remove edge while preserving vertices.

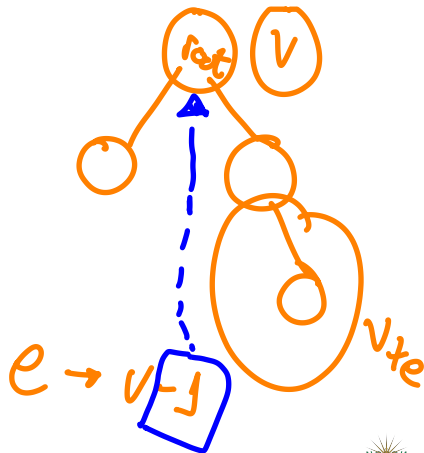
Vertex Fusion

Merge two vertices u and v into a single vertex.



Outline

- 1 Operations on Graphs
- 2 Trees as Graphs
- 3 Algorithms and Analysis
- 4 Summary



Introduction to Trees

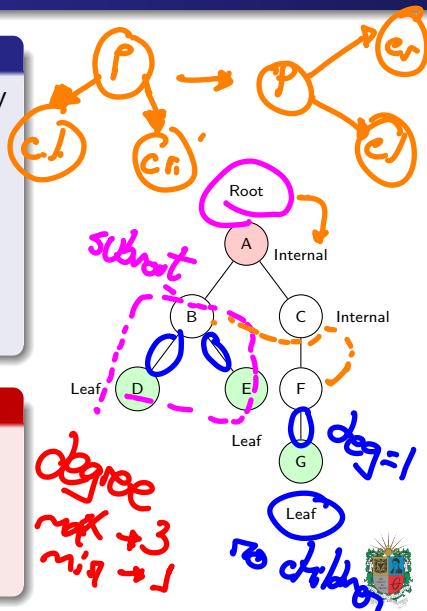
Definition

A tree is a connected acyclic graph. Key properties:

- $|E| = |V| - 1$
- Unique path between any two vertices
- Minimum connected graph structure

Tree Structure

- Root: Designated starting vertex
- Leaves: Vertices with degree 1
- Internal nodes: Vertices with degree ≥ 2

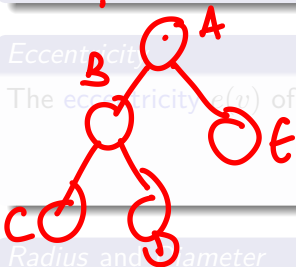


Distance Metrics in Trees

Distance between Vertices

The distance $d(u, v)$ is the length of the unique path from u to v .

Property: In trees, distance is well-defined and unique.



$$d(A, B) = 1$$

$$d(A, C) = 2$$

$$d(A, E) = 1$$

$$d(B, E) = 2 \rightarrow$$

Function

common
ancestors

Radius and Diameter

- **Radius:** $r(G) = \min_v e(v)$
- **Diameter:** $d(G) = \max_v e(v)$



Distance Metrics in Trees

Distance between Vertices

The **distance** $d(u, v)$ is the *length of the unique path* from u to v .

Property: In trees, distance is **well-defined and unique**.

Eccentricity

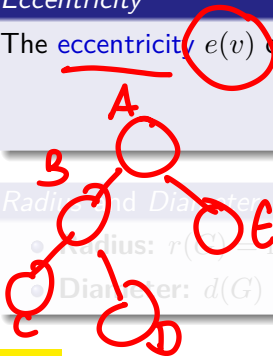
The **eccentricity** $e(v)$ of a vertex v :

$$e(v) = \max_{u \in V} d(v, u)$$

Radius and Diameter

• **Radius:** $r(G) = \min_v e(v)$

• **Diameter:** $d(G) = \max_v e(v)$



$$\begin{aligned} e(A) &= 2 \\ e(B) &= 1 \\ e(C) &= 0 \\ e(D) &= 0 \end{aligned}$$

Diameter = 3
Radius = 2



Distance Metrics in Trees

Distance between Vertices

The **distance** $d(u, v)$ is the *length of the unique path* from u to v .

Property: In trees, distance is **well-defined and unique**.

Eccentricity

The **eccentricity** $e(v)$ of a vertex v :

$$e(v) = \max_{u \in V} d(v, u)$$

Radius and Diameter

- **Radius:** $r(G) = \min_v e(v)$
- **Diameter:** $d(G) = \max_v e(v)$

clustering



Graph Center

Definition

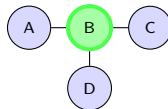
The **graph center** is the set of vertices with **minimum eccentricity**:

$$C(G) = \{v \in V \mid e(v) = r(G)\}$$

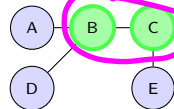
Properties

- **1 or 2 centers** per tree
- If 2: they are *adjacent*
- **Minimize** max eccentricity

1 Center



2 Centers



Outline

- 1 Operations on Graphs
- 2 Trees as Graphs
- 3 Algorithms and Analysis**
- 4 Summary



Algorithms for Graph Operations

Operation Complexity

Operation	Time	Space
Union	$O(V + E)$	$O(V + E)$
Intersection	$O(V + E)$	$O(V + E)$
Vertex deletion	$O(E)$	$O(1)$
Edge deletion	$O(1)$	$O(1)$
Vertex fusion	$O(E)$	$O(1)$

→ memory

constant → $\sim O(1)$

Distance Calculation Algorithm

- BFS/DFS from vertex v
- Track maximum distance encountered
- Return eccentricity $e(v)$
- Complexity: $O(|V| + |E|)$ per vertex

Algorithms for Graph Operations

Operation Complexity

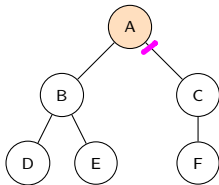
Operation	Time	Space
Union	$O(V + E)$	$O(V + E)$
Intersection	$O(V + E)$	$O(V + E)$
Vertex deletion	$O(E)$	$O(1)$
Edge deletion	$O(1)$	$O(1)$
Vertex fusion	$O(E)$	$O(1)$

Distance Calculation Algorithm

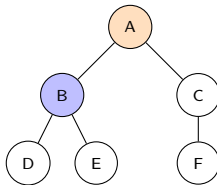
- BFS/DFS from vertex v
- Track maximum distance encountered
- Return eccentricity $e(v)$
- Complexity: $O(|V| + |E|)$ per vertex

Calculating Eccentricity: *Step-by-Step*

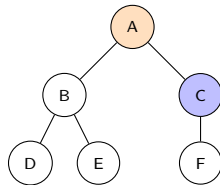
Compute $e(v)$ for each vertex



$$e(A) = 2$$



$$e(B) = 2$$



$$e(C) = 3$$

Key Insight

Eccentricity is the **maximum distance** from v to any other vertex. For each starting vertex, run BFS and track the farthest vertex reached.



Algorithm Pseudocode

Eccentricity

```

function Eccentricity( $G, v$ )
  dist  $\leftarrow$  BFS( $G, v$ )
  max_dist  $\leftarrow$  0
  for each  $u \in V$  do
    if dist[ $u$ ] > max_dist
      max_dist  $\leftarrow$  dist[ $u$ ]
    end if
  end for
  return max_dist
end function

```

All Eccentricities

```

function AllEccentricities( $G$ )
  ecc  $\leftarrow$  []
  for each  $v \in V$  do
    ecc[ $v$ ]  $\leftarrow$ 
      Eccentricity( $G, v$ )
  end for
  return ecc
end function

```



Algorithm Pseudocode

Eccentricity

```

function Eccentricity( $G, v$ )
  dist  $\leftarrow$  BFS( $G, v$ )
  max_dist  $\leftarrow$  0
  for each  $u \in V$  do
    if dist[ $u$ ] > max_dist
      max_dist  $\leftarrow$  dist[ $u$ ]
    end if
  end for
  return max_dist
end function

```

All Eccentricities

```

function AllEccentricities( $G$ )
  ecc  $\leftarrow$  []
  for each  $v \in V$  do
    ecc[ $v$ ]  $\leftarrow$  Eccentricity( $G, v$ )
  end for
  return ecc
end function

```

5 $v \rightarrow$ 5 BFS



Optimization Strategies

Distance Matrix Caching

For repeated queries, precompute and cache all pairwise distances. Trade memory for speed: $O(|V|^2)$ space for $O(1)$ query time (Floyd-Warshall preprocessing).

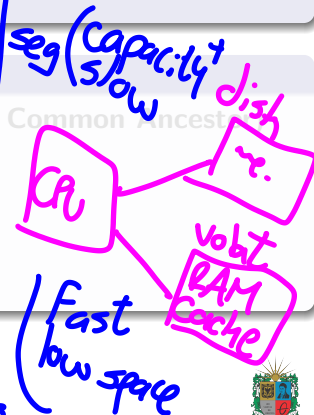


Tree Property Exploitation

Since trees have unique paths (LCA (Lowest Common Ancestor) techniques:

- Precompute binary lifting in $O(|V| \log |V|)$ time
- Query distance in $O(\log |V|)$ time
- Useful for many queries on static trees

cache
+ window lifetime



Optimization Strategies

Distance Matrix Caching

For **repeated queries**, precompute and cache all pairwise distances. Trade memory for speed: $O(|V|^2)$ space for $O(1)$ query time (Floyd-Warshall preprocessing).

Tree Property Exploitation

Since trees have **unique paths**, use **LCA (Lowest Common Ancestor)** techniques:

- **Precompute** binary lifting in $O(|V| \log |V|)$
- Query distance in $O(\log |V|)$ time
- Useful for **many queries** on static trees

no change
a lot

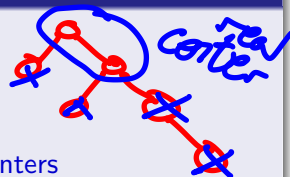


Finding Tree Centers

Algorithm Overview

Leaf Stripping Method:

- ① Start with all vertices as candidates
- ② Iteratively remove leaves (degree 1 vertices)
- ③ When $|V| \leq 2$ vertices remain: these are the centers

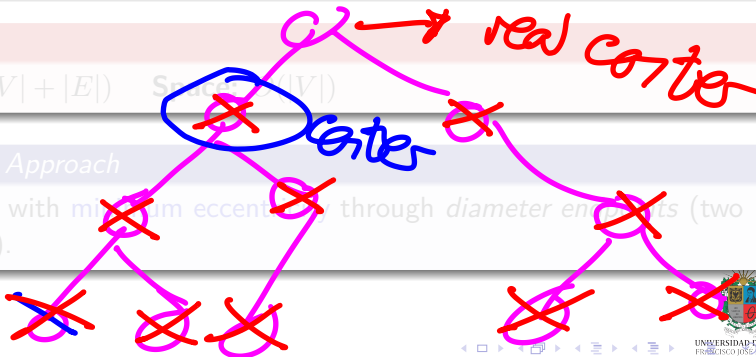


Complexity

Time: $O(|V| + |E|)$ Space: $O(|V|)$

Alternative Approach

Find vertex with minimum eccentricity through diameter endpoints (two BFS passes).



Finding Tree Centers

Algorithm Overview

Leaf Stripping Method:

- ① Start with **all vertices** as candidates
- ② *Iteratively remove* leaves (degree 1 vertices)
- ③ When $|V| \leq 2$ vertices remain: **these are the centers**

Complexity

Time: $O(|V| + |E|)$ **Space:** $O(|V|)$

Alternative Approach

Find vertex with **minimum eccentricity** through diameter endpoints (two BFS passes).

→ *most expensive*



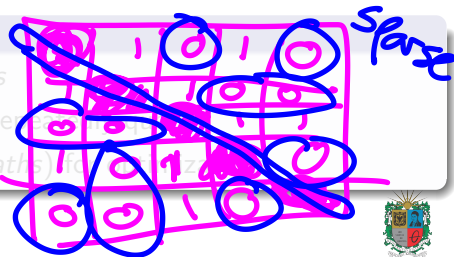
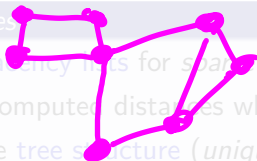
Complexity Analysis Framework

Factors Affecting Performance

- **Graph density:** *Sparse* vs. *dense* affects algorithm selection
- **Representation:** *Adjacency list* vs. *matrix* impacts operation costs
- **Operation frequency:** *Trade-off* between *preprocessing* and *query time*
- **Memory constraints:** *Space-time trade-off* in practical systems

Best Practices

- Use adjacency lists for *sparse* trees
- Cache computed distances when re
- Leverage tree structure (*unique paths*) for



Complexity Analysis Framework

Factors Affecting Performance

- **Graph density:** *Sparse vs. dense* affects algorithm selection
- **Representation:** Adjacency list vs. matrix impacts operation costs
- **Operation frequency:** *Trade-off* between preprocessing and query time
- **Memory constraints:** *Space-time trade-off* in practical systems

Best Practices

- Use adjacency lists for sparse trees
- Cache computed distances when repeatedly queried
- Leverage tree structure (*unique paths*) for optimizations



Outline

- 1 Operations on Graphs
- 2 Trees as Graphs
- 3 Algorithms and Analysis
- 4 Summary



Topics Covered

✓ Graph Operations

Union, Intersection, Ring sum, and structural modifications.

✓ Trees as Graphs

Specialized graph structures with unique properties.

✓ Distance, Eccentricity, and Center

Metrics for analyzing tree topology and finding central nodes.

✓ Algorithms and Analysis

Efficient computational methods and complexity characterization.

graph as a set of nodes and edges



Topics Covered

✓ Graph Operations

Union, Intersection, Ring sum, and *structural modifications*.

✓ Trees as Graphs

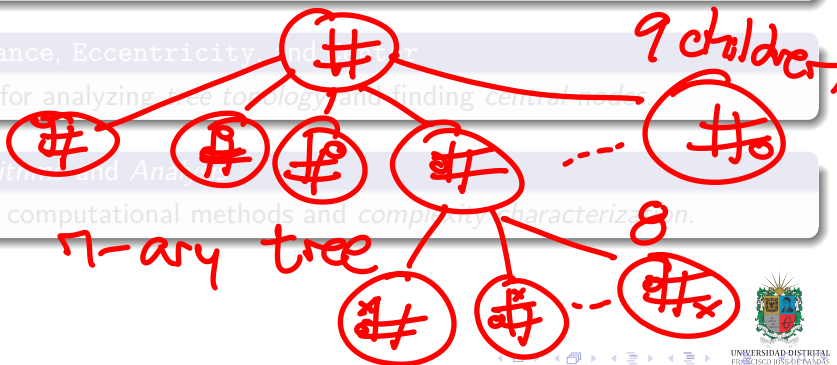
Specialized graph structures with **unique properties**.

✓ Distance, Eccentricity and Center

Metrics for analyzing tree topology and finding central nodes

✓ Algorithms and Analysis

Efficient computational methods and *complexity characterization*.



Topics Covered

✓ Graph Operations

Union, Intersection, Ring sum, and *structural modifications*.

✓ Trees as Graphs

Specialized graph structures with **unique properties**.

✓ Distance, Eccentricity, and Center

Metrics for analyzing tree topology and finding *central nodes*.

✓ *Algorithms and Analysis*

Efficient computational methods and *complexity characterization*.



Topics Covered

✓ Graph Operations

Union, Intersection, Ring sum, and *structural modifications*.

✓ Trees as Graphs

Specialized graph structures with **unique properties**.

✓ Distance, Eccentricity, and Center

Metrics for analyzing *tree topology* and finding *central nodes*.

✓ Algorithms and Analysis

Efficient computational methods and *complexity characterization*.



Outline

- 1 Operations on Graphs
- 2 Trees as Graphs
- 3 Algorithms and Analysis
- 4 Summary



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-ii>

